DEVIXO SOLUTIONS

STUDENT MANAGEMENT SYSTEM

Web Development Internship Report

Student Name: Abubakar Siddique

Internship Organization: Devixo Solutions

Internship Domain: Web Development

Task Number: Task 03 — Full Stack Student Management System

Technologies: React.js, Node.js, Express.js, MongoDB

GitHub Repository: https://github.com/abubakar136/student-management-system

Report Date: 25 October 2026

---

1. Project Overview

The Student Management System is a full-stack web application developed to manage student records through a web-based interface. It uses React.js for the frontend, Node.js and Express.js for the backend, and MongoDB for storing data.

The application allows authenticated users to add, view, search, update, and delete student records. It also provides dashboard statistics, recent registrations, form validation, and feedback during application operations.

2. Objective

The main objective of this project is to develop a user-friendly web application that simplifies student record management.

The project aims to:

- Provide secure user registration and login.
- Allow users to manage student records.
- Display student information and dashboard statistics.
- Store records in a database.
- Validate form inputs and handle errors.
- Provide a responsive and organized interface.

3. Problem Statement

Managing student records manually can be time-consuming and make it difficult to find, update, and maintain information. A computerized system can make these tasks easier by keeping records organized in one place.

The Student Management System addresses this problem by providing a web-based platform for managing student information efficiently.

4. Main Features

4.1 User Authentication

The system provides registration, login, and logout functionality. Authentication protects the application’s restricted features from unauthorized access.

4.2 Student Management

Users can add new student records, view existing records, search for students, edit information, and delete records when necessary.

4.3 Dashboard

The dashboard displays student-related information, including total student statistics and recent registrations.

4.4 Search Functionality

Users can search student records to find the required information more easily.

4.5 Form Validation

The application checks required fields before submitting data and handles duplicate roll numbers to help maintain accurate records.

4.6 Loading Indicators and Messages

The interface provides loading indicators and action messages to inform users about ongoing operations and their results.

4.7 Database Persistence

Student records are stored in MongoDB through the backend, allowing data to remain available after the application is restarted.

4.8 Responsive Interface

The frontend uses CSS styling to provide an organized interface that is intended to work across different screen sizes.

5. Technologies Used

React.js: Used to build the frontend interface and dashboard.

Node.js: Provides the JavaScript runtime for the backend.

Express.js: Used to create backend routes and APIs.

MongoDB: Used to store student and application data.

JavaScript: Implements application logic and interactions between the frontend and backend.

HTML and CSS: Used for page structure and visual styling.

Git and GitHub: Used for source-code version control and repository hosting.

Visual Studio Code: Used to write, run, and debug the application.

6. Implementation Steps

Step 1: Project Setup

A project folder was created with separate frontend and backend directories. The frontend was initialized using React and Vite, while the backend was developed using Node.js and Express.js.

Step 2: Backend Development

Express.js was used to create API endpoints for application operations. The backend manages requests from the frontend and communicates with the database.

Step 3: Database Integration

MongoDB was connected to the backend to store and retrieve student records.

Step 4: Authentication

Registration and login functionality was implemented. Protected backend operations require authentication, and logout functionality allows users to end their session.

Step 5: Student Record Management

The application was developed to support adding, viewing, searching, updating, and deleting student records.

Step 6: Frontend Integration

The React frontend was connected to the Express backend so that users could interact with stored student data through the interface.

Step 7: Dashboard Development

Dashboard statistics and recent student registrations were added to provide a quick overview of the records.

Step 8: Validation and User Feedback

Required-field validation, duplicate roll-number handling, loading indicators, and action messages were added to improve usability.

Step 9: Styling and Testing

CSS was used to style the interface. The application was checked for important operations, including authentication and student record management.

Step 10: GitHub Repository

The project source code and README documentation were uploaded to GitHub.

Repository: https://github.com/abubakar136/student-management-system

7. System Workflow

The application follows this general workflow:

1. The user opens the React frontend in a web browser.
2. The user registers or logs in.
3. The frontend sends requests to the Express backend.
4. The backend processes requests and communicates with MongoDB.
5. MongoDB stores or returns the requested information.
6. The backend sends a response to the frontend.
7. The frontend updates the dashboard or displays the result to the user.

System Architecture:

User → React Frontend → Node.js and Express Backend → MongoDB Database

8. Code Screenshots and Explanations

This section should contain screenshots of the actual source code from the project.

```jsx
import { useCallback, useEffect, useState } from 'react'
import './App.css'
import Login from './Login'
import Register from './Register'

function App() {
  const [authData, setAuthData] = useState(() => {
    try {
      const saved = localStorage.getItem('studentMSAuth')
      return saved ? JSON.parse(saved) : null
    } catch {
      localStorage.removeItem('studentMSAuth')
      return null
    }
  })

  const [authPage, setAuthPage] = useState('login')

  const [apiMessage, setApiMessage] = useState(
    'Connecting to backend...'
  )
  const [students, setStudents] = useState([])
  const [searchTerm, setSearchTerm] = useState('')
  const [connectionError, setConnectionError] = useState(false)
  const [studentsError, setStudentsError] = useState(false)

  const [formData, setFormData] = useState({
    name: '',
    rollNumber: '',
    department: '',
    semester: '3'
```

```jsx
import { useState } from 'react'

function Login({ onLogin, onShowRegister }) {
  const [email, setEmail] = useState('')
  const [password, setPassword] = useState('')
  const [error, setError] = useState('')
  const [loading, setLoading] = useState(false)

  const handleSubmit = async (event) => {
    event.preventDefault()
    setError('')
    setLoading(true)

    try {
      const response = await fetch(
        'http://localhost:5000/api/auth/login',
        {
          method: 'POST',
          headers: {
            'Content-Type': 'application/json',
          },
          body: JSON.stringify({ email, password }),
        }
      )

      const data = await response.json()

      if (!response.ok) {
        throw new Error(data.message || 'Login failed.')
      }
```

.

Explanation: The React frontend displays student information and allows users to interact with the system through forms, search functionality, and action buttons.

Figure 2: Express Backend

Screenshot: Insert a screenshot of an actual Express route used to manage or retrieve student records.

Explanation: The backend receives frontend requests, processes application logic, and returns responses to the client.

Figure 3: MongoDB Integration

server > JS db.js > ...

```
1
2   import mongoose from "mongoose";
3   const connectDB = async () => {
4     try {
5       await mongoose.connect(process.env.MONGO_URI);
6       console.log("MongoDB Connected Successfully");
7     } catch (error) {
8       console.error("MongoDB Connection Failed:", error.message);
9       process.exit(1);
10    }
11  };
12
13  export default connectDB;
14
```

Explanation: MongoDB integration allows the application to save and retrieve student records.

Note: Insert real screenshots from your source code. The screenshot descriptions above are not substitutes for the required images.

9. Output and Result Screenshots

This section should contain screenshots showing the running application.

Figure 4: Login Page

StudentMS

Welcome Back

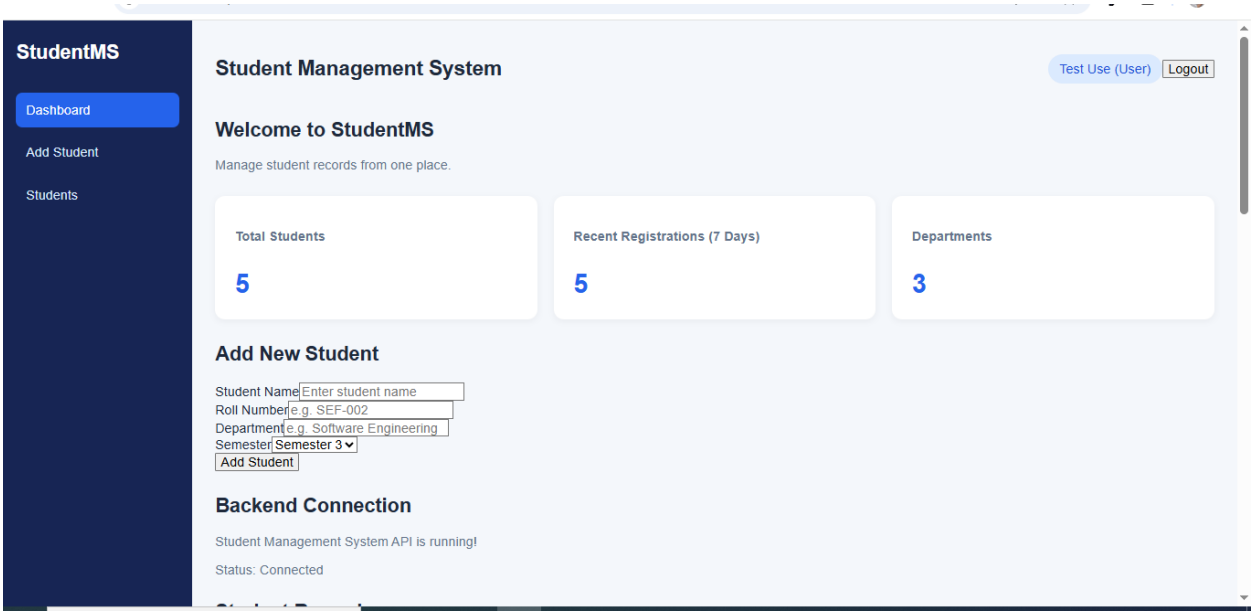
Log in to manage student records.

Email Enter your email Password Enter your password Login

Don't have an account? Register

Explanation: The login interface allows registered users to access the application.

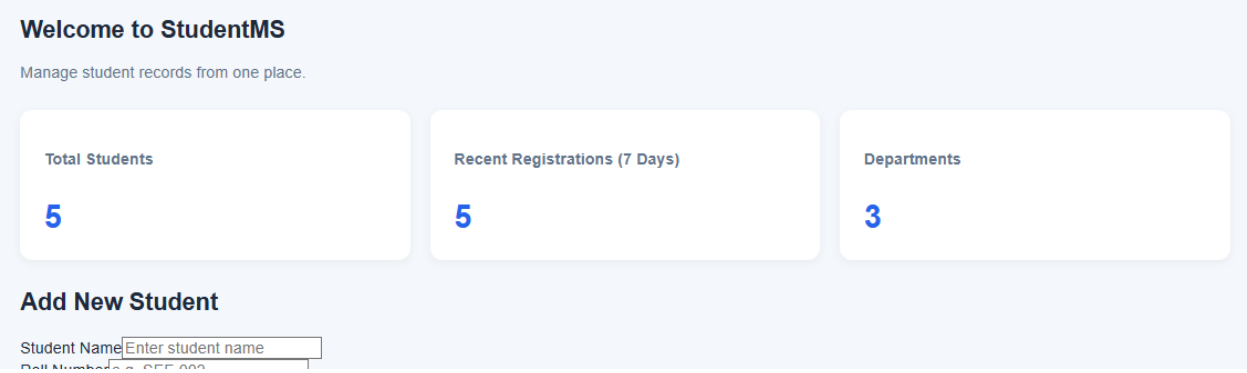
Figure 5: Dashboard



.

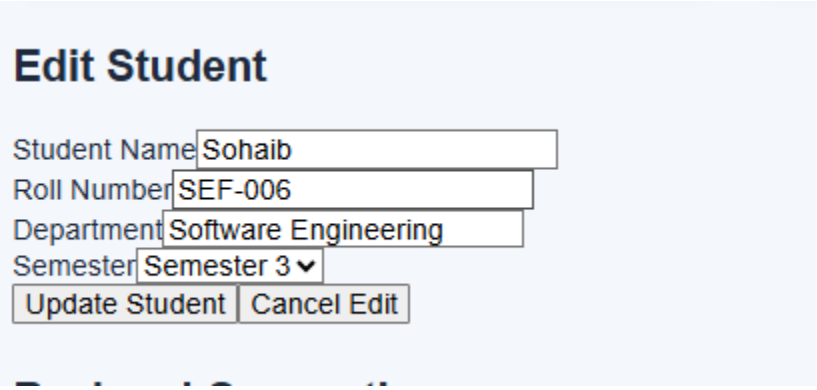
Explanation: The dashboard provides an overview of student records and displays relevant statistics.

Figure 6: Student Records



Explanation: Users can view student records and search for the required information.

Figure 7: Add or Edit Student



Explanation: The form allows users to enter student information and update existing records.

10. Challenges Faced

During development, several technical areas required attention:

- Connecting the React frontend with the Express backend.
- Configuring the MongoDB database connection.
- Implementing authentication and protected API operations.
- Handling student record creation, updating, and deletion.
- Validating form inputs and preventing duplicate roll numbers.
- Displaying loading indicators and useful messages.
- Configuring Git and GitHub while keeping environment variables private.

These tasks provided practical experience in full-stack web development and debugging.

11. Testing

The application was checked against its main functionality.

Test Area	Expected Result
User registration and login	Users can register and log in
Authentication protection	Restricted operations require authentication
Add student	A valid record can be added
View students	Stored records can be displayed
Search students	Matching records can be found
Edit student	Existing information can be updated
Delete student	A selected record can be deleted
Form validation	Missing required information is handled
Database persistence	Records remain stored in MongoDB
Loading and messages	Users receive feedback during operations

The table lists the intended checks for the main features. Record final pass/fail results only after testing each item in the running application.

12. Conclusion

The Student Management System demonstrates the development of a full-stack web application using React.js, Node.js, Express.js, and MongoDB. It brings student record management into a single interface and supports authentication, CRUD operations, searching, dashboard statistics, validation, and database integration.

Working on this project provided practical experience in frontend development, backend APIs, database connectivity, application testing, and GitHub version control. The project can be improved further by adding features such as pagination, more advanced role-based permissions, and deployment to a production environment.

13. GitHub Repository

Project Repository:
https://github.com/abubakar136/student-management-system

The repository contains the project source code and documentation. The README file should provide the setup instructions, required environment configuration, and commands needed to run the frontend and backend.

End of Report